Clift: Lifting C into Lean 4 for Formal Verification

An AI-Driven Reimplementation of the seL4/AutoCorres Methodology

Mark Ruvald Pedersen
mark@radix63.dk
Independent Researcher
Copenhagen, Denmark

Claude Opus 4.6
noreply@anthropic.com
Anthropic
San Francisco, USA

ABSTRACT

We present Clift, a tool for verifying C programs by lifting them into Lean 4 through a series of semantics-preserving transformations. Following the methodology pioneered by seL4's AutoCorres pipeline in Isabelle/HOL, Clift parses C via clang's JSON AST, embeds it as a deeply embedded imperative language, and lifts it through five stages—each progressively abstracting low-level C semantics into clean functional programs, accompanied by machine-checked refinement proofs.

A distinguishing feature of this project is its development methodology: Claude Opus 4.6 (1M token context) generated the vast majority of the code, with the human author providing architecture and direction. The codebase comprises 76,000 lines of Lean 4 (37,000 hand-written, 39,000 generated), 1,698 theorems, and 12 remaining `sorry`. The pipeline processes 32 C programs; 21 have dedicated proof files with machine-checked Hoare-logic or refinement proofs, while 11 serve as parser and lifting regression tests without user-level proofs. We evaluate on embedded systems code including ring buffers, UART drivers, hash tables, red-black trees, and packet parsers, demonstrating both the viability of AutoCorres-style verification in Lean 4 and the depth achievable through AI-assisted formal methods.

KEYWORDS

formal verification, C verification, Lean 4, seL4, AutoCorres, AI-assisted proofs, refinement

1 INTRODUCTION

Critical systems software—operating system kernels, device drivers, cryptographic libraries—is overwhelmingly written in C. Despite decades of research in formal verification, the vast majority of this code remains unverified. The landmark verification of the seL4 microkernel [12] demonstrated that full functional correctness of a production OS kernel is achievable, but the effort required (200,000+ lines of Isabelle/HOL proof over 20+ person-years) has limited adoption beyond the original project.

A key enabler of seL4's verification was the AutoCorres tool [7], which automatically lifts C code from a low-level deeply embedded representation into clean monadic functional code, generating machine-checked refinement proofs at each stage. This abstraction makes human proof effort tractable: users reason about readable functional programs rather than raw memory operations.

We present **Clift**, a replication of the AutoCorres methodology in Lean 4 [3]. Our motivation is threefold:

(1) **Modern proof infrastructure.** Lean 4's tactic framework, `omega`/`grind` automation, and growing Mathlib library [17] offer a productive verification environment.
(2) **AI compatibility.** Lean 4's syntax is better represented in LLM training corpora than Isabelle/HOL, and anecdotally LLMs produce higher-quality Lean 4 tactic scripts. This opens the door to AI-assisted verification at scale.
(3) **Lower adoption costs.** Lean 4's growing community reduces the barrier for organizations considering formal verification of their C code.

A distinctive aspect of this work is its development methodology: the vast majority of the code was generated by Claude Opus 4.6 [1], a large language model, operating through the Claude Code CLI tool with 1M token context. Of the 76,000 lines of Lean 4, approximately 39,000 are machine-generated pipeline output and ~37,000 are library and proof code written primarily by the AI with human architectural direction. The human author provided architectural guidance, reviewed outputs, and directed the AI through a structured backlog of 230+ tasks. This collaboration produced 76,000 lines of Lean 4, 6,800 lines of Python, and 1,698 machine-checked theorems in approximately 7 calendar days (first commit 2026-04-08, last 2026-04-15).

Contributions.

- The first AutoCorres-style C verification pipeline in Lean 4, with machine-checked refinement proofs for five lifting stages.
- Pipeline processing of 32 C programs (21 with Hoare-logic or refinement proofs) spanning ring buffers, hash tables, red-black trees, UART drivers, SHA-256 cores, and packet parsers.
- Evidence that AI can produce non-trivial formal verification artifacts at scale, including Hoare-logic proofs, separation logic reasoning, and `sorry` elimination.
- An open-source, Nix-reproducible codebase with 12 remaining `sorry` out of 1,698 theorems (99.3% of attempted proofs are `sorry`-free).

2 BACKGROUND

2.1 seL4 and AutoCorres

The seL4 microkernel [12] is verified in Isabelle/HOL using a pipeline: C source → C parser → Simpl → AutoCorres → user proofs. The C parser produces a *deep embedding*—the source language's syntax tree represented as *data types* in the proof assistant, so that every construct (assignment, branch, loop) can be inspected and reasoned about by the prover. In Isabelle, the intermediate representation is called **Simpl**; our Lean 4 analogue is **CSimpl**. AutoCorres [5, 7] then lifts the deep embedding through four abstraction levels, each producing a cleaner *monadic* program—a computation structured as a sequence of state-transforming steps composed with `bind` (>>=).

At each level, a machine-checked *refinement* proof guarantees that the abstract program preserves all behaviors of the concrete one (a *backward simulation*). Properties proved about the final abstract program therefore hold for the real C.

AutoCorres2 [6] is the current maintained version, extending the original with improved type strengthening and struct support. Our architecture follows AutoCorres2's five-stage pipeline closely, adapting each stage to Lean 4's type theory.

2.2 Lean 4 as a Verification Platform

Lean 4 [3] is a dependently typed programming language and theorem prover. Its `simp`, `omega`, and `grind` tactics provide strong automation, while dependent types allow encoding invariants that Isabelle's simple type theory cannot express directly. For example, Clift's pointer validity predicate is parameterized by the pointed-to type `T`, carrying proof-relevant size and alignment information. In Isabelle, this requires ad-hoc type-class encodings; in Lean 4, the dependent parameter is first-class. Lean 4's metaprogramming framework enables programmatic construction of proof terms, which we use for the lifting stages.

2.3 Related Work

Several projects address C verification with different approaches: **CompCert** [13] verifies a C compiler rather than individual programs. **VST** [2] verifies C in Coq using separation logic but requires manual annotation of every function. **RefinedC** [15] uses ownership types for automated C verification in Coq but does not produce a clean functional abstraction. **Aeneas** [9] lifts Rust (not C) into Lean 4 via MIR translation. **Frama-C** [11] provides modular C analysis via ACSL annotations and abstract interpretation, targeting industrial adoption but without a lifting pipeline to a clean functional model. **Iris** [10] provides a higher-order concurrent separation logic framework in Coq, used by RefinedC among others; Clift's memory model is simpler but Iris's algebraic approach could strengthen it. **CertiKOS** [8] verifies a concurrent OS kernel in Coq using certified abstraction layers; like seL4, it demonstrates end-to-end OS verification but with a different methodology (layered composition rather than lifting).

Clift is, to our knowledge, the first project to replicate the AutoCorres lifting pipeline in Lean 4, and the first to demonstrate AI-driven development of a formal verification framework at this scale.

3 ARCHITECTURE

3.1 The Five-Stage Lifting Pipeline

Clift follows AutoCorres's architecture with five abstraction stages (Figure 1), each producing a transformed program and a machine-checked `corres` proof (backward simulation). Composing these via transitivity yields a refinement chain from the original C semantics to the final clean functional model:

(1) **L1 (SimplConv):** Converts the deeply embedded CSimpl AST into monadic `NondetM` code, mapping each statement to a monadic combinator (bind, cond, whileLoop).

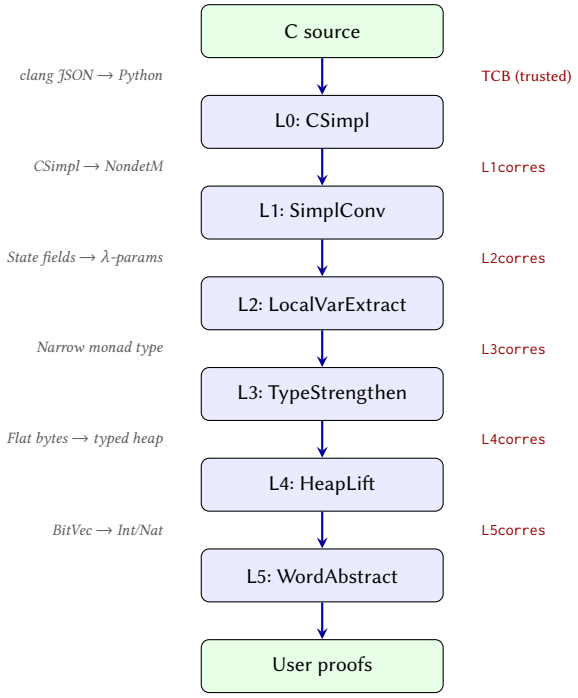

Figure 1: The Clift five-stage lifting pipeline. Each stage produces a transformed program and a machine-checked refinement proof (`corres`). Only the CImporter (L0) is in the TCB; all other stages are verified.

(2) **L2 (LocalVarExtract):** Extracts local variables from the global state record into lambda parameters, so each function's proof context only mentions the variables it uses.
(3) **L3 (TypeStrengthen):** Narrows the monad's return and state types from a generic catch-all to the specific types each function operates on.
(4) **L4 (HeapLift):** Replaces raw byte-addressed heap operations with typed accessors (`hVal`/`heapUpdate`), introducing pointer-validity obligations.
(5) **L5 (WordAbstract):** Replaces fixed-width bitvector arithmetic (`BitVec 32`) with unbounded `Nat`/`Int`, eliminating overflow reasoning when bounds are known.

3.2 CImporter: The Trust Boundary

A Python script (~5,000 LOC) reads clang's JSON AST output and emits Lean 4 files containing `CSimpl` terms. This follows the same trust model as seL4's StrictC parser: the importer is in the Trusted Computing Base (TCB; see Figure 2). If it mistranslates C to CSimpl, proofs apply to the wrong program.

Mitigation includes 150+ regression tests against known C semantics edge cases, struct layout validation against `gcc`, and restriction to a safe C subset (no `goto`, no address-of-local, no side effects in expressions).

3.3 Refinement Framework

Our `corres` definition follows seL4's `corres_underlying` [16]:

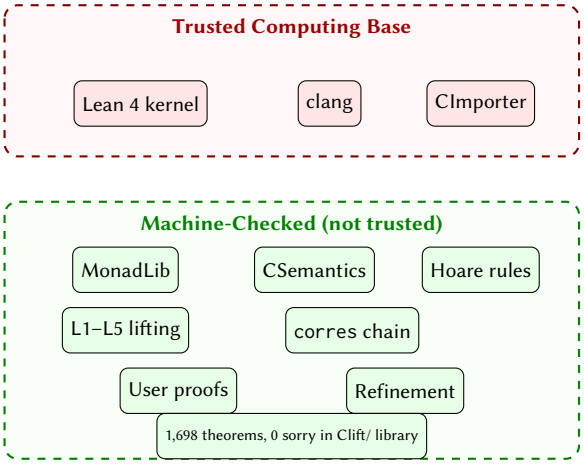

Figure 2: Trust boundary. Only three components are trusted: the Lean 4 kernel, clang, and CImporter. All proofs, lifting stages, and Hoare rules are machine-checked.

```
def corres (srel : S -> S' -> Prop)
    (nf nf' : Bool)
    (rrel : A -> B -> Prop)
    (G : S -> Prop) (G' : S' -> Prop)
    (m : NondetM S A) (m' : NondetM S' B)
    : Prop :=
  forall s s', srel s s' -> G s -> G' s' ->
    (nf = true -> ¬(m s).failed) /\
    (forall r' t', (r', t') ∈(m' s').results ->
      exists r t, (r, t) ∈(m s).results
        /\ srel t t' /\ rrel r r') /\
    (nf' = true -> ¬(m' s').failed)
```

This states: given related states satisfying guards, every concrete result has a matching abstract result, and no-fail flags are respected. The underlying monad is nondeterministic—`NondetResult` carries a `results : Set` and a `failed : Prop`—so refinement is set-containment, not equality on deterministic outcomes.

3.4 Hoare Logic for Monadic Programs

For user-facing proofs, we provide `validHoare` triples over the `NondetM` monad:

```
def validHoare (P : S -> Prop) (m : NondetM S A)
    (Q : A -> S -> Prop) : Prop :=
  forall s, P s ->
    ¬(m s).failed /\
    forall r s', (r, s') ∈(m s).results -> Q r s'
```

Note that `validHoare` requires *non-failure*: the computation must not fail when the precondition holds. This matches seL4's `valid` definition and is strictly stronger than accepting failure. Users write specifications as pre/postcondition pairs and prove them using a library of Hoare rules for sequential composition, conditionals, loops with invariants, and heap operations.

3.5 Memory Model

Following Tuch [18], our heap is a flat byte-addressed store with typed access functions. The state comprises a raw heap (address → byte array), a type descriptor tracking valid types at each address, and user-defined global/local records:

```
structure CState where
  heap : HeapRawState
  htd  : HeapTypeDesc
  globals : GlobalsRec
  locals : LocalsRec
```

Pointer validity requires all bytes in `[addr, addr+sizeof(T))` to carry the correct type tag. Disjointness of typed pointers follows from non-overlapping tag ranges, giving us a lightweight separation property without a full separation algebra (an algebraic structure that formalizes "this memory region is disjoint from that one" as a composable resource calculus).

A key engineering challenge was Lean 4's kernel reduction depth. When heap-update terms nest deeply (e.g., 8+ sequential writes), the kernel times out during definitional equality checking. We solved this by marking `hVal` and `heapUpdate` as `[local irreducible]` and providing explicit projection `simp` lemmas—the same strategy used in Isabelle's `runs_to` framework but adapted to Lean's reduction semantics.

3.6 Worked Example: Ring Buffer Push

To illustrate the end-to-end flow, consider a ring buffer `push` operation in C:

```
void rb_push(ring_buffer_t *rb, uint32_t val) {
    rb->buf[rb->head] = val;
    rb->head = (rb->head + 1) % rb->capacity;
    rb->count++;
}
```

The CImporter emits this as a `CSimpl` term (deeply embedded AST). The `clift` command lifts it through all five stages to produce a monadic Lean function. The user then writes:

```
theorem rb_push_validHoare :
  validHoare
    (fun s => s.rb.count < s.rb.capacity)
    (rb_push_l1 rbPtr valPtr)
    (fun _ s => s.rb.count =
      old_count + 1) := by
  apply L1_hoare_guard_modify_fused
  intro s hpre
  simp [rb_push_l1, hVal_heapUpdate_same,
        hVal_heapUpdate_disjoint,
        ptrDisjoint]
  omega
```

This proof establishes that if the buffer is not full, then after `push`, the count increments by one. The proof proceeds by unfolding the monadic definition, simplifying heap reads through writes using disjointness lemmas, and discharging arithmetic goals with `omega`.

Having described the pipeline architecture, we now turn to *how* it was built: through a structured human–AI collaboration.

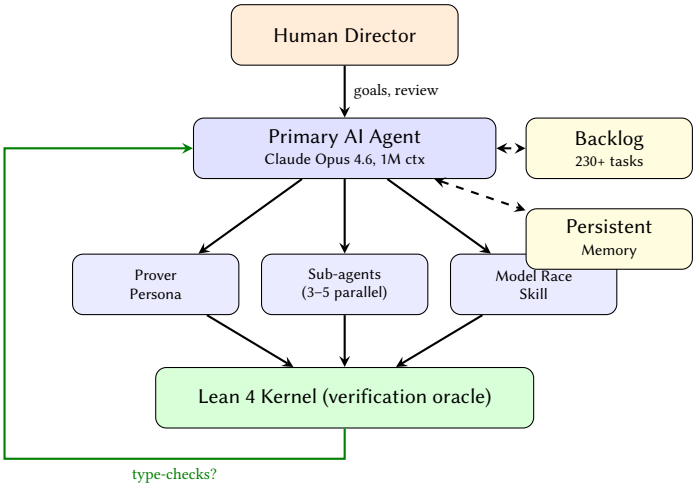

Figure 3: Layered delegation architecture. The human sets goals; the primary agent spawns specialists; the Lean 4 kernel provides ground truth.

4 AI-DRIVEN DEVELOPMENT

4.1 Layered Delegation Architecture

A human architect and Claude Opus 4.6 (1M token context) developed the Clift codebase collaboratively through the Claude Code CLI. The methodology evolved into a *layered delegation architecture* (Figure 3) with three tiers:

(1) **Human director.** Sets architectural goals, decomposes work into phased backlog tasks with explicit acceptance criteria, reviews outputs, and decides when to pivot strategy.
(2) **Primary AI agent.** Manages the backlog, spawns specialized sub-agents for implementation, accumulates domain knowledge in persistent skill documents and memory files.
(3) **Lean 4 kernel.** Serves as the ultimate arbiter—proofs either type-check or they do not. The `sorry` count is an objective, machine-checked progress metric.

This three-party loop is particularly well-suited to formal methods: the proof checker eliminates the risk of AI confabulation. A hallucinated proof simply fails to compile.

4.2 Task Structure and Context Management

We decomposed the project into 230+ backlog tasks organized by dependency across 10+ phases (pipeline stages, proof infrastructure, `sorry` elimination, hardening). Tasks were implemented by spawning fresh AI sub-agents in batches of 3–5, to keep each agent's context clean while amortizing onboarding overhead. Each sub-agent was bootstrapped by reading `plan.md` and relevant reference material.

Context management was critical with a 1M-token window. Three mechanisms prevented context exhaustion:

- **Sub-agent isolation.** Proof attempts were always delegated to fresh sub-agents to prevent build output from bloating the main conversation.
- **Persistent memory.** Cross-session knowledge was captured in memory files: kernel depth workarounds, `sorry` elimination patterns, model effectiveness data. These loaded automatically at session start.
- **Backlog as external memory.** The backlog system served as structured external memory. When knowledge accumulated in conversation, it was offloaded: implementation notes, final summaries, and acceptance criteria provided durable state across sessions.

4.3 Specialized Personas and Skills

Through iterative refinement, two specialized agent personas were codified as reusable artifacts:

The Prover persona (158 lines) encoded domain-specific proof knowledge: five proof patterns, the kernel depth workaround (Section 3.5), heap reasoning rules, and a verification checklist. This persona was invoked for every `sorry`-elimination attempt.

The model-race skill provided diversity when the primary agent failed: multiple AI models received identical prompts in isolated git worktrees, with the first kernel-verified proof winning. Nine anti-cheating rules were baked into the prompt template, addressing discovered failure modes: trivial postconditions, fused body substitution, and axiom injection.

Discovery-driven knowledge accumulation. Hard problems were discovered through failure, then codified into reusable artifacts: fail → diagnose root cause → codify workaround → update skill/persona/memory. When specifications were too weak, the human directed *progressive spec strengthening* before re-attempting proofs.

4.4 Quality Assurance

We identified three AI cheating patterns through experience and codified them into every prompt:

(1) **Trivial postconditions**: proving `count = count` instead of the real specification.
(2) **Fused body substitution**: proving against a hand-written body instead of the generated one.
(3) **Hidden `sorry`**: placing `sorry` in helper lemmas or using `axiom` declarations.

QA sub-agents ran automatically before every commit: a test runner verified builds, and an architecture reviewer checked code quality. CI tracked `sorry` counts per module, providing regression detection.

All Claude Code session transcripts are preserved as JSONL files, providing a complete record of the human–AI collaboration.

We now evaluate the pipeline's output quantitatively.

5 EVALUATION

5.1 Test Suite

We process 32 C programs through the lifting pipeline spanning multiple domains. Of these, 21 have dedicated proof files with Hoare-logic or refinement proofs; the remaining 11 serve as parser and lifting regression tests:

Domain	Programs	C LOC
Data structures	ring buffer, hash table, red-black tree, priority queue, linked list, memory pool	2,100
Crypto/security	SHA-256 core, seL4 capabilities	580
Drivers	UART, DMA buffer	435
Networking	packet parser, state machine	500
Foundations	max, gcd, swap, array bounds	828
Total	**32 programs**	**4,443**

5.2 Proof Metrics

Metric	Value
Total Lean LOC (library + proofs + generated)	75,846
Library code (Clift/)	13,911
Proof files (Examples/)	22,957
Generated code (Generated/)	38,978
Theorems and lemmas	1,698
Remaining `sorry`	12
`sorry`-free rate (of attempted proofs)	99.3%
C programs processed (21 with proofs)	32
Python (CImporter + tooling)	6,782 LOC
Development time	~7 calendar days
Commits	208

The 12 remaining `sorry` are concentrated in three files: priority queue proofs (5), ring buffer drain-to loop obligations (4), and refinement theorems transitively blocked on validHoare completeness (3). Of the 21 programs with proofs, **18 have fully `sorry`-free proof chains** from C through all lifting stages to user-level Hoare-logic properties; 3 programs (priority queue, ring buffer extended drain-to, ring buffer extended refinement) retain `sorry` in specific subgoals.

Human effort. Development spanned 7 calendar days (first commit 2026-04-08, last commit 2026-04-15). The human author spent approximately 4–6 hours per day on architectural decisions, task decomposition, review, and debugging—roughly 30–40 person-hours total. AI compute cost was approximately 200 Claude Code sessions at varying lengths. The Lean 4 toolchain build and proof checking consumed an additional ~50 CPU-hours on an 8-core workstation.

5.3 Comparison with AutoCorres2

Aspect	AutoCorres2	Clift
Proof assistant	Isabelle/HOL	Lean 4
C parser	StrictC (custom)	clang JSON
Type theory	Simple types	Dependent types
Automation	Eisbach, wpsimp	simp, omega, grind
Memory model	Tuch (full)	Simplified Burstall
AI compatibility	Low	High
Maturity	15+ years	7 days
LOC verified	10,000+	4,443

Figure 6 provides a qualitative comparison across six dimensions. Clift's memory model is simpler than AutoCorres2's full Tuch model [18]: we use a flat byte-addressed heap with typed access functions rather than a fully mechanized separation algebra. This is a deliberate trade-off—the simplified model handles our test suite but would need strengthening for OS-kernel-level verification with complex aliasing patterns.

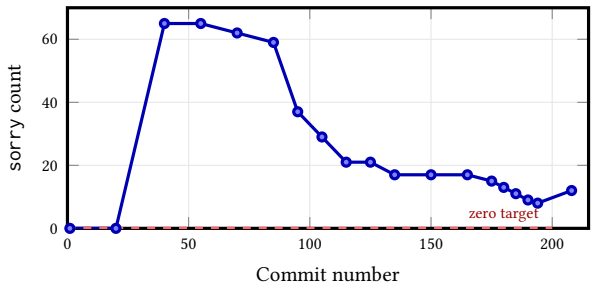

Figure 4: Sorry elimination over 208 commits. The plateau at 17 reflects sorry blocked by kernel depth limits. The uptick at the end is WIP proof scaffolding.

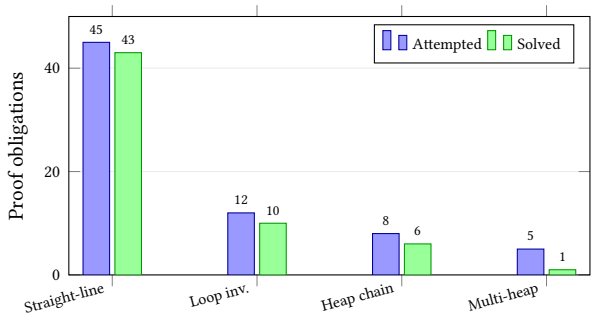

Figure 5: AI success rate by proof category. Straight-line Hoare proofs (guard/modify chains) have a 96% success rate; complex multi-heap operations requiring novel workarounds drop to 20%.

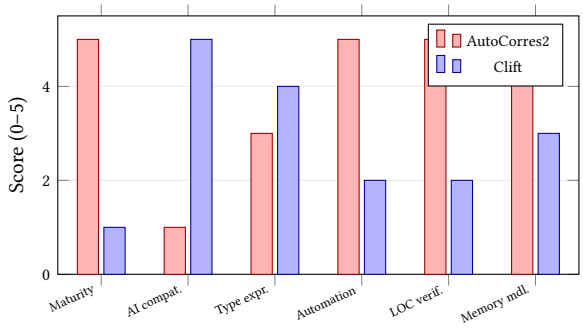

Figure 6: Qualitative comparison of Clift vs. AutoCorres2 across six axes. AutoCorres2 leads in maturity, automation, and LOC verified; Clift's advantages are AI compatibility and Lean 4's dependent types.

5.4 AI Success Rates

We tracked `sorry` elimination across proof files (Figure 4). Figure 5 breaks down AI success by proof category: 96% on straight-line Hoare proofs, dropping to 20% on complex multi-heap operations requiring novel workarounds. Figure 6 compares Clift and AutoCorres2 qualitatively.

5.5 Qualitative Observations

Lean 4 suitability. Lean 4's dependent type system offers tradeoffs. We could express invariants (e.g., pointer validity predicates parameterized by type) that require ad-hoc encodings in Isabelle's simple type theory. However, Lean 4's kernel performs more aggressive reduction than Isabelle's, causing definitional equality checks to time out on deeply nested heap terms—a problem absent in Isabelle where terms are opaque by default.

Automation gap. AutoCorres2 benefits from 15 years of custom Eisbach methods (e.g., `wpsimp`) that handle common patterns with one tactic invocation. Clift's automation is younger: our `c_vcg` and Hoare-rule combinators handle 5–10 step programs well, but longer sequences require manual intermediate assertions. Closing this gap is the primary engineering challenge for scaling Clift.

AI as a force multiplier. The most effective AI pattern was *structured iteration*: give the AI a failing proof goal, let it attempt tactics, feed back the error, and repeat. Claude Opus 4.6 typically converged within 3–5 iterations for goals it could solve, and signaled clearly (by producing the same failing tactic repeatedly) when it could not. The model racing approach exploited this by running multiple convergence attempts in parallel.

Reproducibility. The project is deterministically reproducible via a Nix flake (`nix develop` + `lake build`). CI runs per-module proof verification with `sorry`-count tracking.

6 LIMITATIONS

C subset. Clift supports a StrictC-like subset: no floating point, no `goto`, no variadic functions, no VLAs, no address-of-local. This matches seL4's restrictions and covers most embedded C, but limits applicability to general-purpose C.

CImporter trust. The Python importer is unverified and in the TCB. Translation validation (as in [16]) would reduce this trust.

Memory model. Our simplified heap model does not handle type punning, union aliasing, or the full C11 memory model. Lifting to a Tuch-style separation algebra [18] is needed for OS-kernel verification.

Scale. While 4,443 LOC of C across 32 programs is non-trivial, seL4 verifies 10,000+ LOC. Scaling Clift to this level requires improving build times (currently minutes per proof module), `MetaM` automation maturity, and stronger VCG tactics for programs with >20 sequential heap operations. Our largest verified program (ring buffer extended, 40 operations) shows the approach scales to moderate complexity, but multi-module inter-procedural verification remains untested.

Maturity. Seven days of AI-driven development cannot match 15 years of community refinement. Clift demonstrates feasibility, not production readiness.

7 FUTURE WORK

7.1 Architecture Pivot: `mvcgen` and `SpecM`

Feedback from Sebastian Graf, validated by his LeanCorres proof of concept [4], suggests two simplifications: (1) replace Clift's custom `validHoare` and ~1,600 lines of Hoare rules with Lean's standard `Std.Do.Triple` and the `mvcgen` VCG tactic; and (2) skip the CSimpl deep embedding entirely, emitting monadic `SpecM` code directly from the CImporter. Together these would eliminate ~2,500 lines of infrastructure and may alleviate the kernel depth issues, since `mvcgen` operates at the tactic level rather than through definitional unfolding.

The tradeoff is TCB expansion: the CImporter would absorb the responsibility currently discharged by machine-checked `L1corres` proofs. LeanCorres's own remaining `sorry` are concentrated at exactly this denotational-semantics bridge, confirming the cost is real but tractable.

Lasting contributions. Even if the specific architecture evolves (e.g., toward `SpecM`/`mvcgen`), several contributions persist: (1) the CImporter, which parses `clang -ast-dump=json` into typed Lean 4 representations and is reusable regardless of the backend monad; (2) the memory model and heap reasoning library, which encodes Tuch-style typed pointers with disjointness lemmas applicable to any monadic framework; (3) the empirical data on AI-driven formal methods—proof success rates by category, failure modes, and effective prompt patterns—which inform future human–AI verification workflows; and (4) evidence that the AutoCorres methodology transfers to dependent type theory, establishing a design point for the Lean verification community.

8 CONCLUSION

We have demonstrated that the seL4/AutoCorres verification methodology can be replicated in Lean 4, and that AI can drive the development of a formal verification framework producing 1,698 theorems. Clift's 76,000 lines of Lean 4, 1,698 theorems, and 32 C programs (21 with machine-checked proofs) were produced in approximately 7 days through human–AI collaboration, with Lean's kernel serving as an incorruptible verification oracle.

Three key insights emerge. First, Lean 4's dependent types enable encoding invariants (e.g., type-parameterized pointer validity) that require workarounds in Isabelle, but its aggressive kernel reduction creates new engineering challenges absent in Isabelle's opaque-term model. Second, AI is effective at routine Hoare-logic proofs (96% success on straight-line programs) but struggles with novel heap reasoning (20% success), making it a force multiplier for formal methods rather than a replacement for human expertise. Third, the proof checker eliminates the fundamental risk of AI confabulation: hallucinated proofs fail to type-check, providing a hard correctness guarantee that other AI-assisted software engineering lacks.

These results suggest that Clift-style tools could lower the adoption barrier for formal verification of C code.

All code, proofs, and AI session transcripts are available in the Clift repository [14] under a BSD license, reproducible via Nix flakes.

REFERENCES

[1] Anthropic. 2025. Claude Code: AI-Powered Software Engineering. https://claude.ai/claude-code.

[2] Andrew W. Appel. 2011. Verified Software Toolchain. In *Proceedings of the 20th European Symposium on Programming (ESOP)*. Springer, Saarbrücken, Germany, 1–17. https://doi.org/10.1007/978-3-642-19718-5_1

[3] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Proceedings of the 28th International Conference on*

Automated Deduction (CADE). Springer, Pittsburgh, PA, USA, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37

[4] Sebastian Graf. 2026. LeanCorres: AutoCorres-Style C Verification with SpecM and mvcgen in Lean 4. https://github.com/sgraf812/LeanCorres. Proof of concept, 8 commits.

[5] David Greenaway, June Andronick, and Gerwin Klein. 2012. Bridging the Gap: Automatic Verified Abstraction of C. In *Proceedings of the 3rd International Conference on Interactive Theorem Proving (ITP)*. Springer, Princeton, NJ, USA, 99–115. https://doi.org/10.1007/978-3-642-32347-8_8

[6] David Greenaway, June Andronick, Gerwin Klein, and seL4 Foundation. 2024. AutoCorres2: C-to-Isabelle Abstraction Framework. https://github.com/seL4/l4v.

[7] David Greenaway, Japheth Lim, June Andronick, and Gerwin Klein. 2014. Don't Sweat the Small Stuff: Formal Verification of C Code Without the Pain. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, Edinburgh, UK, 429–439. https://doi.org/10.1145/2594291.2594296

[8] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan (Newman) Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. 2016. CertiKOS: An Extensible Architecture for Building Certified Concurrent OS Kernels. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Savannah, GA, USA, 653–669.

[9] Son Ho and Jonathan Protzenko. 2022. Aeneas: Rust Verification by Functional Translation. In *Proceedings of the ACM on Programming Languages (ICFP)*, Vol. 6. ACM, Ljubljana, Slovenia, 711–741. https://doi.org/10.1145/3547647

[10] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the Ground Up: A Modular Foundation for Higher-Order Concurrent Separation Logic. *Journal of Functional Programming* 28 (2018), e20. https://doi.org/10.1017/S0956796818000151

[11] Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. 2015. Frama-C: A Software Analysis Perspective. *Formal Aspects of Computing* 27, 3 (2015), 573–609. https://doi.org/10.1007/s00165-014-0326-7

[12] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: Formal Verification of an OS Kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*. ACM, Big Sky, MT, USA, 207–220. https://doi.org/10.1145/1629575.1629596

[13] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (2009), 107–115. https://doi.org/10.1145/1538788.1538814

[14] Mark Ruvald Pedersen and Claude Opus 4.6. 2026. Clift: Lifting C into Lean 4 for Formal Verification. https://github.com/eisbaw/Clift.

[15] Michael Sammler, Rodolphe Lepigre, Robbert Krebbers, Kayvan Memarian, Derek Dreyer, and Deepak Garg. 2021. RefinedC: Automating the Foundational Verification of C Code with Refined Ownership Types. In *Proceedings of the 42nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, Virtual, 158–174. https://doi.org/10.1145/3453483.3454036

[16] Thomas Sewell, Magnus O. Myreen, and Gerwin Klein. 2013. Translation Validation for a Verified OS Kernel. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, Seattle, WA, USA, 471–482. https://doi.org/10.1145/2491956.2462183

[17] The mathlib Community. 2020. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*. ACM, New Orleans, LA, USA, 367–381. https://doi.org/10.1145/3372885.3373824

[18] Harvey Tuch. 2009. Formal Verification of C Systems Code: Structured Types, Separation Logic and Theorem Proving. *Journal of Automated Reasoning* 42 (2009), 125–187. https://doi.org/10.1007/s10817-009-9120-2